COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/14/2026	**DATE SUBMITTED:**	10/16/2026
NAME:	Libo-on, Gian Carlo G. Antonio, Christine Mae P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. **ALGORITHMIC STRATEGIES**

1. PROCEDURE OUTPUT

ILO A:

Strategy	Algorithm	Analysis
Recursion	**Binary Search**	Binary search applies recursion by diving a problem into half of the exact problem to solve it quicker. This method applies recursion since it automatically calls the function itself. Basically, the iterative function of binary search which allows it to half itself is recursion.
Brute Force	**Bubble Sort**	Brute forcing finds the target by trying all the possible solutions. Much like how bubble sorting happens. It traverses through an array one by one until it finds the target or the array is sorted already.
Backtracking	**Depth First Search**	DFS goes to the deepest part of the tree then applies recursion where it checks each node one by one. Backtracking is the same, it also applies recursion from the deepest point to the highest point which is basically why DFS and Backtracking are the same.
Greedy	**Selection Sort**	A greedy algorithm only does the locally optimal process of a problem to solve it. Same as a selection sort which selects the smallest/largest value in an unsorted array and puts in a sorted array. This works but it is not always the most optimal process.
Divide-and-Conquer	**Merge Sort**	A merge sort divides the array in half, until only two values are left to compare to each other then they will swap. The same can be said in the divide and conquer approach

Table 12-1: Algorithmic Strategies and Examples

Screenshot

```cpp
#include <iostream>
#include <algorithm>

 // we will initialize the elements to -1 ( -1 means, not solved it yet )
int getMinSteps(int n,int memo[]){
    if ( n == 1 ){
    std::cout << "Best Case Found\n";
    return 0; // base case
}
    if( memo[n] != -1 ){ // we have solved it already :)
    std::cout << "Case Already Solved\n";
    return memo[n];
}
    int r = 1 + getMinSteps( n - 1, memo ); // '-1' step . 'r' will contain the optimal answer finally
    if( n%2 == 0 ) r = std::min( r , 1 + getMinSteps( n / 2, memo )) ; // '/2' step
    if( n%3 == 0 ) r = std::min( r , 1 + getMinSteps( n / 3, memo )) ; // '/3' step
    memo[n] = r ; // save the result. If you forget this step, then it's the same as plain recursion.
    return r;
}

int main() {
    int n = 10;
    int memo[100];
    std::fill(memo, memo + 100, -1);
    getMinSteps(n, memo);
    return 0;
}
```

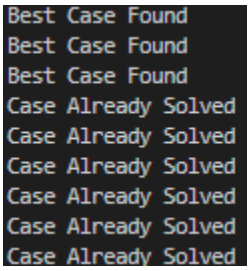
	
Analysis	**Memoization allows the storing and caching of data. In the given code snippet, the program needs to find the best case in solving until n reaches one. The first three numbers 1,2,3 found the best cases because the conditions allowed it to be able to recurse only once. Moreover, the other numbers were able to be solved. The time complexity of this code snippet is O(n) with memoization applied. However, it's $O(3^n)$ if memoization is not used.**

Table 12-2. Memoization Implementation

Screenshot

```
#include <iostream>
#include <algorithm>

int getMinSteps ( int n ){
    int* dp = new int[n+1];
    int i;
    dp[1] = 0; // trivial case
    for( i = 2 ; i <= n ; i ++ ){
        dp[i] = 1 + dp[i-1];
        if(i%2==0) dp[i] = std::min( dp[i] , 1+ dp[i/2] );
        if(i%3==0) dp[i] = std::min( dp[i] , 1+ dp[i/3] );
    }
    int retVal = dp[n];
    delete[] dp;
    return retVal;
}

int main(){
    std::cout << getMinSteps(10) << '\n';
    return 0;
}
```

	
Analysis	This approach is the bottoms up approach. Unlike starting from the top to the bottom, we start from the bottom going up. The time complexity is 2 to n. Unlike the previous approach where we need to declare an array, bottom-up implementation removes that process. It completely removes the risk of stack overflow and recursion overhead.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B:

PSEUDOCODE

```
FUNCTION Paths (matrix, m, n, cost):
  IF m < 0 OR n < 0 OR cost < 0 THEN
    RETURN 0
  END IF

  IF m == 0 AND n == 0 THEN
    IF cost == matrix[0][0] THEN
      RETURN 1
    ELSE
      RETURN 0
    END IF
  END IF

  Rem_cost = cost - mat[m][n]

  RETURN Paths (matrix, m-1, n, rem_cost) + Paths (matrix, m, n-1, rem_cost)
END
```

SOLVING BY HAND

SOLUTION 1

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

SOLUTION 2

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

ALL SOLUTIONS

PATH	SOLUTION
R R R D D D	$4 + 7 + 1 + 6 + 9 + 2 + 3 = 32$
R R D R D D	$4 + 7 + 1 + 3 + 9 + 2 + 3 = 29$
R R D D R D	$4 + 7 + 1 + 3 + 1 + 2 + 3 = 21$
R R D D D R	$4 + 7 + 1 + 3 + 1 + 7 + 3 = 26$
R D R R D D	$4 + 7 + 7 + 3 + 9 + 2 + 3 = 35$
R D R D R D	$4 + 7 + 7 + 3 + 1 + 2 + 3 = 27$
R D R D D R	$4 + 7 + 7 + 3 + 1 + 7 + 3 = 32$
R D D R R D	$4 + 7 + 7 + 8 + 1 + 2 + 3 = 32$
R D D R D R	$4 + 7 + 7 + 8 + 1 + 7 + 3 = 37$
R D D D R R	$4 + 7 + 7 + 8 + 1 + 7 + 3 = 37$
D R R R D D	$4 + 6 + 7 + 3 + 9 + 2 + 3 = 34$
D R R D R D	$4 + 6 + 7 + 3 + 1 + 2 + 3 = 26$
D R R D D R	$4 + 6 + 7 + 3 + 1 + 7 + 3 = 31$
D R D R R D	$4 + 6 + 7 + 8 + 1 + 2 + 3 = 31$

DRDRDR	$4 + 6 + 7 + 8 + 1 + 7 + 3 = 36$
DRDDRR	$4 + 6 + 7 + 8 + 1 + 7 + 3 = 36$
DDRRRD	$4 + 6 + 3 + 8 + 1 + 2 + 3 = 27$
DDRRDR	$4 + 6 + 3 + 8 + 1 + 7 + 3 = 32$
DDRDRR	$4 + 6 + 3 + 8 + 1 + 7 + 3 = 32$
DDDRRR:	$4 + 6 + 3 + 7 + 1 + 7 + 3 = 31$

CODE

```
#include <iostream>
#include <vector>

//matrix size 4x4
//definition of variables
#define M 4
#define N 4

//Top-down recursion (matrix, m, n, cost)
int Paths(int matrix[M][N], int m, int n, int cost) {
    //Base case 1
    //define out of bounds as program termination
    if (m < 0 || n < 0 || cost < 0) {
        return 0;
    }

    //Base Case 2
    //define starting cell as (0,0)
    if (m == 0 && n == 0) {
        return (cost == matrix[0][0]) ? 1 : 0;
    }
```

```
    //count paths by moving up (m-1, n) and left (m, n-1)
    int remainingCost = cost - matrix[m][n];
    return Paths(matrix, m - 1, n, remainingCost) +
           Paths(matrix, m, n - 1, remainingCost);
}

int main() {
    //4x4 matrix definition
    int matrix[M][N] = {
        { 4, 7, 1, 6 },
        { 6, 7, 3, 9 },
        { 3, 8, 1, 2 },
        { 7, 1, 7, 3 }
    };
    //target cost initialization
    int cost = 25;
    int alt_cost = 26;

    //Start from destination cell (M-1, N-1)
    int totalPaths = Paths(matrix, M - 1, N - 1, cost);
    int totalPathsAlt = Paths(matrix, M - 1, N - 1, alt_cost);

    std::cout << "The total number of paths with cost " << cost
              << " is " << totalPaths << std::endl;

    std::cout << "The total number of paths with cost " << alt_cost
              << " is " << totalPathsAlt << std::endl;

    return 0;
}
```

OUTPUT

```
The total number of paths with cost 25 is 0
The total number of paths with cost 26 is 2
```

ANALYSIS	
ANTONIO	**LIBO-ON**
This uses a recursive algorithm strategy to count paths. It counts the right and down paths from (0,0) to (m,n) which cell values add up to cost. The output for cost 25 is 0 as there was no cost found when the matrix was recursed through. With cost 26, there were two, which followed paths R-R-D-D-D-R and D-R-R-D-R-D. The running time is exponential which is $O(2^{m+n})$ because the same cells gets recomputed over and over.	This approach uses a top-down process of recursion. It started from the top left down to the bottom right since the movement of the recursion can only go down or right. Unfortunately, even after 20 attempts, there is no exact path that leads to 25 costs which is why we decided to stick with 26 costs. The time complexity of the program was about $O(2^{M+N})$ since each cell creates 2 recursive calls each time. And the space complexity is about O(M+N).

CONCLUSION	
ANTONIO	**LIBO-ON**
In this activity, the covered topic was about Algorithm strategies. These strategies help with speed and accuracy of result, but varies in how they break up a problem. Recursion and divide and conquer divide problems into small parts, brute force and backtracking techniques look for solutions exhaustively, while greedy techniques make decisions one by one. Greedy algorithms are quite quick in decision making, but they do not always result in an optimal solution. These are under dynamic programming.	Overall, I learned a lot about this activity especially how dynamic programming works and why it is important especially for solving problems such as the ones in the supplementary activities. I also learned a lot about algorithms and how they affect the efficiency of the program in terms of usability and problem solving capability. I think overall, this activity is fun and successful.

REFERENCES

https://www.geeksforgeeks.org/dsa/difference-between-recursion-and-dynamic-programming/
https://www.geeksforgeeks.org/dsa/number-of-paths-with-exactly-k-coins/

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, Pseudocode assistance, Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I strictly used it only as a basis and reference**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.